

Reviewer Pack — Matroid Girth of Monotone DNF Implies k-CLIQUE Circuit Size ...

Entry c18d42c448ab · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 09:26:30 UTC

Matroid Girth of Monotone DNF Implies k-CLIQUE Circuit Size Lower Bound

Entry ID: c18d42c448ab **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 09:26:30 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Monotone Circuit Size for k-CLIQUE

Statement:

Let F be a monotone DNF formula representing the k -CLIQUE function on n -vertex graphs. Let $M(F)$ be the simple binary matroid whose ground set is the set of terms of F , and whose independent sets are subsets of pairwise edge-disjoint terms. Then the girth of $M(F)$ — the size of the smallest circuit (minimal dependent set) in $M(F)$ — is at least k . Furthermore, if a monotone DNF has matroid girth $< k$, it cannot compute k -CLIQUE on all n -vertex graphs for $n \geq 3k$.

Rationale (proposer's reasoning):

The girth in this matroid measures the smallest number of edge-disjoint but combinatorially overlapping terms required to create a dependency, which corresponds to unavoidable overcounting in clique detection. High girth forces structural complexity in the DNF that reflects the necessity of large support for covering all k -cliques without false positives. This connects extremal combinatorics in matroids to proof complexity in monotone circuits.

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cd223f8964c2e39e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_prime(n):
        if n <= 1:
            return False
        for i in range(2, int(math.sqrt(n)) + 1):
            if n % i == 0:
                return False
        return True

    def generate_primes(count):
        primes = []
        num = 2
        while len(primes) < count:
            if is_prime(num):
                primes.append(num)
            num += 1
        return primes

    def gcd(a, b):
        while b:
            a, b = b, a % b
        return a

    def lcm(a, b):
        return abs(a * b) // gcd(a, b)

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
```

```

C = [[0] * p for _ in range(m)]
for i in range(m):
    for j in range(p):
        for k in range(n):
            C[i][j] += A[i][k] * B[k][j]
return C

def gaussian_elimination(A, b):
    n = len(b)
    M = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(M[j][i]) > abs(M[max_row][i]):
                max_row = j
        M[i], M[max_row] = M[max_row], M[i]
        factor = M[i][i]
        for j in range(n):
            M[i][j] /= factor
        for j in range(n):
            if i != j:
                factor = M[j][i]
                for k in range(n+1):
                    M[j][k] -= factor * M[i][k]
    return [M[i][-1] for i in range(n)]

def is_independent(matroid, subset):
    for i in range(len(subset)):
        for j in range(i+1, len(subset)):
            if not matroid[subset[i]][subset[j]]:
                return False
    return True

def find_circuits(matroid):
    n = len(matroid)
    circuits = []
    for r in range(2, n+1):
        for subset in itertools.combinations(range(n), r):
            if not is_independent(matroid, subset):
                circuits.append(subset)
    return circuits

def generate_monotone_dnf(n, k):
    terms = [random.sample(range(n*(n-1)//2), random.randint(1, n)) for _ in range(k)]
    dnf = []
    for term in terms:
        clause = []
        for edge in term:
            u, v = divmod(edge, n)
            if u < v:
                clause.append((u, v))
        dnf.append(clause)
    return dnf

def evaluate_dnf(dnf, assignment):
    for clause in dnf:

```

```

        if all(assignment[u] == 1 and assignment[v] == 1 for u, v in clause):
            return True
        return False

def generate_matroid(dnf):
    n = len(dnf)
    matroid = [[False] * n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            if any((i, j) in clause or (j, i) in clause for clause in dnf):
                matroid[i][j] = True
                matroid[j][i] = True
    return matroid

def girth(matroid):
    circuits = find_circuits(matroid)
    if not circuits:
        return float('inf')
    return min(len(circuit) for circuit in circuits)

n = 20
k = 3
dnf = generate_monotone_dnf(n, k)
matroid = generate_matroid(dnf)
girth_value = girth(matroid)

metric_name = "Matroid Girth"
metric_value = girth_value
instances_tested = 1
conjecture_holds = girth_value >= k
counterexample = "" if conjecture_holds else f"DNF with girth < {k}"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or generate_primes(30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")

```

```

elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"girth < {k}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

: 'DNF with girth < 3'}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Matroid Girth', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
RESULT: INCONCLUSIVE insufficient data

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test instances have $n=1$ (implied by `instances_tested=1`) which is too small to establish scaling behavior. The metric ‘matroid girth’ is trivially bounded below by 2 for any non-trivial matroid, making the conjecture’s bound vacuous. The counterexample cited ($\text{girth} < 3$) is actually the minimal possible girth, suggesting the metric is saturated.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test instances use $n=1$ (too small) and matroid girth ≥ 2 is trivial, making the conjecture’s bound vacuous. Counterexample is minimal girth, not violating conjecture’s conditions. | next: Test with $n \geq 3k$ for $k=3$ ($n \geq 9$) and verify girth ≥ 3 for k -CLIQUE DNFs

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	cerebras	qwen-3-235b-a22b-ins	0	0	21379
2	preregistration	ollama_remote	qwen3:8b	0	0	24021
3	novelty	ollama_remote	qwen3:8b	0	0	21634

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	qwen3:8b	0	0	13645
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16630
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18753
7	critic	ollama_remote	qwen3:8b	0	0	99880
8	judge	ollama_remote	qwen3:8b	0	0	18197

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 234139 ms total latency. Provider mix: {'cerebras': 1, 'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/c18d42c448ab.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c18d42c448ab.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c18d42c448ab.tar.gz` (if generated)